

Vec3

**Indian Institute Of Information Technology
Kottayam**

Pratyush Nanda

Aniket Singh

Siddhant Nema

// Smallest Prime Factor

```

const int MAXN = 1e6;
vector<int> spf(MAXN + 1);
// vector<int> pri;
void sieve() {
    for(int i = 0; i ≤ MAXN; i++) {
        spf[i] = i;
    }
    for(int i = 2; i ≤ MAXN; i++) {
        if(spf[i] == i) {
            // pri.push_back(i);
            for(int j = 2 * i; j ≤ MAXN; j
+= i) {
                spf[j] = min(spf[j], i);
            }
        }
    }
}

```

// Prime Factorization

```

vector<int> fvec(int x) {
    vector<int> f;
    while(x ≠ 1) {
        f.push_back(spf[x]);
        x /= spf[x];
    }
}

```

```

    }
    return f;
}
map<int, int> fmap(int x) {
    map<int, int> f;
    while(x ≠ 1) {
        f[spf[x]]++;
        x /= spf[x];
    }
    return f;
}
// String Tokenizer
vector<string> tokenize(string s, string del
= " ") {
    int start, end = -1 * del.size();
    vector<string> strv;
    do {
        start = end + del.size();
        end = s.find(del, start);
        strv.push_back(s.substr(start, end -
start));
    }
    while (end ≠ -1);
    return strv;
}

```

```

}
// Modulus Functions
const int MOD = 1e9 + 7;
inline int gcdext(int a, int b, int* p, int*
q) {
    if(a == 0) {
        *p = 0;
        *q = 1;
        return b;
    }
    int c, d;
    int g = gcdext(b % a, a, &c, &d);
    *p = d - (b / a) * c;
    *q = c;
    return g;
}
inline int modsum(int a, int b) {
    return ((a + b) % MOD + MOD) % MOD;
}
inline int modpro(int a, int b) {
    return (((a % MOD) * (b % MOD)) % MOD +
MOD) % MOD;
}
inline int modexp(int a, int n) {

```

```

    if(n == 0) {
        return 1;
    }
    int b = modexp(a, n / 2);
    b = modpro(b, b);
    if(n & 1) {
        b = modpro(a, b);
    }
    return (b % MOD + MOD) % MOD;
}
inline int modinv(int a) {
    int b, c;
    int g = gcdext(a, MOD, &b, &c);
    if(g != 1) {
        cerr << "ERROR: GCD DOESN'T EXIST!!!
VERIFY CODE!!!" << endl;
        return 0;
    }
    return (b % MOD + MOD) % MOD;
}
// Manacher Algorithm
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";

```

```

vector<int> p(n + 2);
int l = 1, r = 1;
for(int i = 1; i ≤ n; i++) {
    p[i] = max(0, min(r - i, p[l + (r - i)]));
    while(s[i - p[i]] == s[i + p[i]]) {
        p[i]++;
    }
    if(i + p[i] > r) {
        l = i - p[i];
        r = i + p[i];
    }
}
return vector<int>(begin(p) + 1, end(p) - 1);
}
vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}

```

```

}
// n - Dimension Vector
template<class... Args>
auto vec(size_t n, Args&&... args) {
    if constexpr(sizeof...(args) == 1)
        return vector(n, args...);
    else
        return vector(n, vec(args...));
}
// Ordered Set PBDS
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define ordered_set tree<int,
null_type, less<int>,
rb_tree_tag, tree_order_statistics_node_update
>
// Prefix Rolling Double Hash
pair<vector<int>, vector<int>>
master_hash(vector<int>& a) {
    const int MOD1 = 1e9 + 7;
    const int MOD2 = 1e9 + 9;
    const int BASE1 = 31;
    const int BASE2 = 37;
}

```

```

    int hash1 = 0, hash2 = 0;
    int power1 = 1, power2 = 1;
    vector<int> prefixHash1((int)a.size()),
    prefixHash2((int)a.size());
    for(int i = 0; i < (int)a.size(); ++i) {
        hash1 = (hash1 + a[i] * power1) %
MOD1;
        hash2 = (hash2 + a[i] * power2) %
MOD2;
        prefixHash1[i] = hash1;
        prefixHash2[i] = hash2;
        power1 = (power1 * BASE1) % MOD1;
        power2 = (power2 * BASE2) % MOD2;
    }
    return {prefixHash1, prefixHash2};
}
// FFT
const double PI = acos(-1);
typedef complex<double> cd;
namespace fft {
    void dft(vector<cd> &a, bool invert) {
        int n = a.size();
        vector<int> rev(n);

```

```

        for(int i = 0, j = 0; i < n; i++) {
            if(i < j) {
                swap(a[i], a[j]);
            }
            int bit = n >> 1;
            while(j & bit) {
                j ^= bit;
                bit >>= 1;
            }
            j ^= bit;
        }
        for(int len = 2; len ≤ n; len <= 1)
        {
            double angle = 2 * PI / len *
(invert ? -1 : 1);
            cd wlen(cos(angle), sin(angle));
            for(int i = 0; i < n; i += len) {
                cd w(1);
                for(int j = 0; j < len / 2;
j++) {
                    cd u = a[i + j], v = a[i
+ j + len / 2] * w;
                    a[i + j] = u + v;

```

```

                a[i + j + len / 2] = u -
v;
                w *= wlen;
            }
        }
    }
    if(invert) {
        for(cd &x : a) x /= n;
    }
}
void idft(vector<cd> &a) {
    dft(a, true);
}
vector<int> convolve(vector<int> const
&a, vector<int> const &b) {
    vector<cd> fa(a.begin(), a.end()),
fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() +
(int)b.size()) n *= 2;
    fa.resize(n);
    fb.resize(n);
    dft(fa, false);
    dft(fb, false);

```

```

    for(int i = 0; i < n; i++) {
        fa[i] *= fb[i];
    }
    idft(fa);
    vector<int> result(n);
    for(int i = 0; i < n; i++) {
        result[i] = round(fa[i].real());
    }
    return result;
}
}

```

// Segment Tree (2N Iterative)

```

template<typename T>
class SegmentTree {
public:
    int n;
    vector<T> tree;
    SegmentTree(vector<T> arr, T id) {
        n = arr.size();
        tree.resize(2 * n, id);
        build(arr);
    }
    void build(vector<T> arr) {
        for(int i = 0; i < n; i++) {

```

```

        tree[n + i] = arr[i];
    }
    for(int i = n - 1; i > 0; i--) {
        tree[i] = tree[i * 2] + tree[i *
2 + 1];
    }
}

void update(int index, T value) {
    index += n;
    tree[index] = value;
    while(index > 1) {
        index /= 2;
        tree[index] = tree[index * 2] +
tree[index * 2 + 1];
    }
}

T query(int left, int right) {
    left += n;
    right += n;
    T result1 = tree[0];
    T result2 = tree[0];
    while(left ≤ right) {
        if(left % 2 == 1) {

```

```

        result1 = result1 +
tree[left];
        left++;
    }
    if(right % 2 == 0) {
        result2 = tree[right] +
result2;
        right--;
    }
    left /= 2;
    right /= 2;
}
return result1 + result2;
}
};

// Disjoint Set Union
class DSU {
public:
    int vtx;
    vector<int> prnt, rank, size;
    DSU(int n) {
        vtx = n;
        make(n);
    }

```

```

void make(int n) {
    prnt.resize(n);
    rank.resize(n, 0);
    size.resize(n, 1);
    for(int i = 0; i < n; i++) {
        prnt[i] = i;
    }
}

int find(int u) {
    if(u == prnt[u]) {
        return u;
    }
    return prnt[u] = find(prnt[u]);
}

void unionRank(int u, int v) {
    u = find(u);
    v = find(v);
    if(u != v) {
        if(rank[u] > rank[v]) {
            swap(u, v);
        }
        prnt[u] = v;
        if(rank[u] == rank[v]) {

```

```

            rank[v]++;
        }
    }
}

void unionSize(int u, int v) {
    u = find(u);
    v = find(v);
    if(u != v) {
        if(size[u] > size[v]) {
            swap(u, v);
        }
        prnt[u] = v;
        size[v] += size[u];
    }
}

};

// Graph
class Graph {
public:
    int n;
    vector<vector<int>> adj;
    vector<bool> vst;
    Graph(int _n) {
        n = _n;

```

```

        adj.assign(n, vector<int>());
        vst.assign(n, false);
    }
    void add(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void reset() {
        vst.assign(n, false);
    }
    void dijkstra(int s) {
        priority_queue<pair<int, int>,
vector<pair<int, int>>, greater<pair<int,
int>>> pq;
        spt[s] = 0;
        pq.push({0, s});
        while(pq.size()) {
            auto [d, u] = pq.top();
            pq.pop();
            if(d > spt[u]) {
                continue;
            }
            for(auto [v, w] : adj[u]) {
                if(spt[u] + w < spt[v]) {

```

```

                spt[v] = spt[u] + w;
                pq.push({spt[v], v});
            }
        }
    }
    void dfs(int u) {
        vst[u] = true;
        for(auto v : adj[u]) {
            if(!vst[v]) {
                dfs(v);
            }
        }
        tso.push_back(u);
    }
    void longestPath(int s) {
        for(int u = 0; u < vtx; u++) {
            if(!vst[u]) {
                dfs(u);
            }
        }
        reverse(tso.begin(), tso.end());
        lpt[s] = 0;
        for(auto u : tso) {

```

```

        if(lpt[u]  $\neq$  -INF(int)) {
            for(auto [v, w] : adj[u]) {
                lpt[v] = max(lpt[v],
lpt[u] + w);
            }
        }
        for(int i = 0; i < vtx; i++) {
            for(auto u : tso) {
                if(lpt[u]  $\neq$  -INF(int)) {
                    for(auto [v, w] : adj[u])
{
                        if(lpt[v] < lpt[u] +
w) {
                            lpt[v] =
INF(int);
                        }
                    }
                }
            }
        }
    };
    // LCA

```

```

const int LOG = 20;
vector<vector<int>> up;
vector<int> depth;
void dfs_lca(int u, int p,
vector<vector<int>> &g) {
    up[0][u] = p;
    for(int i = 1; i < LOG; i++)
        up[i][u] = up[i - 1][ up[i - 1][u] ];
    for(int v : g[u]) {
        if(v == p) continue;
        depth[v] = depth[u] + 1;
        dfs_lca(v, u, g);
    }
}

int lca(int u, int v) {
    if(depth[u] < depth[v]) swap(u, v);
    int k = depth[u] - depth[v];
    for(int i = 0; i < LOG; i++)
        if(k & (1 << i))
            u = up[i][u];
    if(u == v) return u;
    for(int i = LOG - 1; i  $\geq$  0; i--) {
        if(up[i][u]  $\neq$  up[i][v]) {
            u = up[i][u];

```

```

        v = up[i][v];
    }
}
return up[0][u];
}
// Coordinate Compression
template<class T>
vector<int> compress(vector<T> &a) {
    vector<T> v = a;
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()),
v.end());
    vector<int> res(a.size());
    for(int i = 0; i < (int)a.size(); i++)
        res[i] = lower_bound(v.begin(),
v.end(), a[i]) - v.begin();
    return res;
}

```

// Z Algorithm

```

vector<int> zfunc(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {

```

```

        if(i ≤ r) {
            z[i] = min(r - i + 1, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i
+ z[i]]) {
            z[i]++;
        }
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

```

// KMP Algorithm

```

vector<int> pfunc(string s) {
    int n = s.size();
    vector<int> p(n);
    for(int i = 1; i < n; i++) {
        int j = p[i - 1];
        while(j > 0 && s[i] ≠ s[j]) {
            j = p[j - 1];
        }
        if(s[i] == s[j]) {

```

```

        j++;
    }
    p[i] = j;
}
return p;
}
vector<int> kmp(string text, string pat) {
    string s = pat + "#" + text;
    vector<int> p = pfunc(s);
    vector<int> idx;
    int m = pat.size();
    for(int i = m + 1; i < (int)s.size();
i++) {
        if(p[i] == m) {
            idx.push_back(i - 2 * m);
        }
    }
    return idx;
}
// Trie
class Trie {
public:
    vector<array<int, 26>> nxt;
    vector<int> end;

```

```

Trie() {
    nxt.push_back({});
    nxt[0].fill(-1);
    end.push_back(0);
}
void insert(string s) {
    int v = 0;
    for(char c : s) {
        int x = c - 'a';
        if(nxt[v][x] == -1) {
            nxt[v][x] = nxt.size();
            nxt.push_back({});
            nxt.back().fill(-1);
            end.push_back(0);
        }
        v = nxt[v][x];
    }
    end[v]++;
}
bool find(string s) {
    int v = 0;
    for(char c : s) {
        int x = c - 'a';
        if(nxt[v][x] == -1)

```

```

        return false;
        v = nxt[v][x];
    }
    return end[v];
}
};
// Hash Trie
class HashTrie {
public:
    vector<unordered_map<char, int>> nxt;
    vector<int> end;
    HashTrie() {
        nxt.push_back({});
        end.push_back(0);
    }
    void insert(string s) {
        int v = 0;
        for(char c : s) {
            if(!nxt[v].count(c)) {
                nxt[v][c] = nxt.size();
                nxt.push_back({});
                end.push_back(0);
            }
            v = nxt[v][c];
        }
    }
};

```

```

    }
    end[v]++;
}
bool find(string s) {
    int v = 0;
    for(char c : s) {
        if(!nxt[v].count(c))
            return false;
        v = nxt[v][c];
    }
    return end[v];
}
};
// Next Greater Element
vector<int> nge(vector<int> &a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;
    for(int i = 0; i < n; i++) {
        while(!st.empty() && a[st.top()] <
a[i]) {
            res[st.top()] = i;
            st.pop();
        }
    }
}

```

```

        st.push(i);
    }
    return res;
}
// Topological Sort (Kahn)
vector<int> topo_sort(int n,
vector<vector<int>> &g) {
    vector<int> indeg(n), order;
    for(int u = 0; u < n; u++)
        for(int v : g[u])
            indeg[v]++;
    queue<int> q;
    for(int i = 0; i < n; i++)
        if(indeg[i] == 0)
            q.push(i);
    while(!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for(int v : g[u]) {
            if(--indeg[v] == 0)
                q.push(v);
        }
    }
    return order;
}

```

```

}
// Floyd Cycle Detection
int find_cycle_start(vector<int> &nxt, int
src) {
    int slow = src, fast = src;
    do {
        slow = nxt[slow];
        fast = nxt[nxt[fast]];
    } while(slow != fast);
    slow = src;
    while(slow != fast) {
        slow = nxt[slow];
        fast = nxt[fast];
    }
    return slow;
}
// Matrix Exponentiation (2x2)
struct mat {
    long long a[2][2];
};
mat mul(mat x, mat y, long long mod) {
    mat r{};
    for(int i = 0; i < 2; i++)
        for(int j = 0; j < 2; j++)

```

```

        for(int k = 0; k < 2; k++)
            r.a[i][j] = (r.a[i][j] +
x.a[i][k] * y.a[k][j]) % mod;
        return r;
    }
mat mpow(mat a, long long e, long long mod) {
    mat r{{{1,0},{0,1}}};
    while(e) {
        if(e & 1) r = mul(r, a, mod);
        a = mul(a, a, mod);
        e >>= 1;
    }
    return r;
}
// RNG
mt19937_64
rng(chrono::steady_clock::now().time_since_ep
och().count());
long long rnd(long long l, long long r) {
    return uniform_int_distribution<long
long>(l, r)(rng);
}
// Aho-Corasick
struct aho {

```

```

static const int A = 26;
vector<array<int, A>> nxt;
vector<int> link, out;
aho() {
    nxt.push_back({});
    nxt[0].fill(-1);
    link.push_back(0);
    out.push_back(0);
}
void insert(string s) {
    int v = 0;
    for(char c : s) {
        int x = c - 'a';
        if(nxt[v][x] == -1) {
            nxt[v][x] = nxt.size();
            nxt.push_back({});
            nxt.back().fill(-1);
            link.push_back(0);
            out.push_back(0);
        }
        v = nxt[v][x];
    }
    out[v]++;
}

```

```

void build() {
    queue<int> q;
    for(int c = 0; c < A; c++) {
        int u = nxt[0][c];
        if(u  $\neq$  -1) q.push(u);
        else nxt[0][c] = 0;
    }
    while(!q.empty()) {
        int v = q.front(); q.pop();
        out[v] += out[link[v]];
        for(int c = 0; c < A; c++) {
            int u = nxt[v][c];
            if(u  $\neq$  -1) {
                link[u] =
nxt[link[v]][c];
                q.push(u);
            } else {
                nxt[v][c] =
nxt[link[v]][c];
            }
        }
    }
}

int match(string s) {

```

```

    int v = 0, res = 0;
    for(char c : s) {
        v = nxt[v][c - 'a'];
        res += out[v];
    }
    return res;
}

};

// Heavy Light Decomposition
vector<int> parent, depth, heavy, head, pos,
sz;
int cur_pos;
int dfs_hld(int v, vector<vector<int>> &g) {
    sz[v] = 1;
    int mx = 0;
    for(int u : g[v]) {
        if(u == parent[v]) continue;
        parent[u] = v;
        depth[u] = depth[v] + 1;
        int usz = dfs_hld(u, g);
        sz[v] += usz;
        if(usz > mx) {
            mx = usz;
            heavy[v] = u;

```

```

    }
}
return sz[v];
}
void decompose(int v, int h,
vector<vector<int>> &g) {
    head[v] = h;
    pos[v] = cur_pos++;
    if(heavy[v]  $\neq$  -1)
        decompose(heavy[v], h, g);
    for(int u : g[v]) {
        if(u == parent[v] || u == heavy[v])
            continue;
        decompose(u, u, g);
    }
}
// Persistent Segment Tree
struct pst {
    struct node {
        int l, r;
        long long val;
    };
    vector<node> t;
    int n;

```

```

pst(int n) : n(n) {
    t.push_back({0, 0, 0}); // dummy
}
int new_node(int l, int r, long long val)
{
    t.push_back({l, r, val});
    return t.size() - 1;
}
int build(int l, int r) {
    if(l == r)
        return new_node(0, 0, 0);
    int m = (l + r) / 2;
    int L = build(l, m);
    int R = build(m + 1, r);
    return new_node(L, R, 0);
}
int upd(int v, int l, int r, int pos,
long long val) {
    if(l == r)
        return new_node(0, 0, val);
    int m = (l + r) / 2;
    int L = t[v].l, R = t[v].r;
    if(pos  $\leq$  m)

```

```

        L = upd(L, l, m, pos, val);
    else
        R = upd(R, m + 1, r, pos, val);
    return new_node(L, R, t[L].val +
t[R].val);
}
long long qry(int v, int l, int r, int
ql, int qr) {
    if(v == 0 || qr < l || r < ql)
        return 0;
    if(ql ≤ l && r ≤ qr)
        return t[v].val;
    int m = (l + r) / 2;
    return qry(t[v].l, l, m, ql, qr)
        + qry(t[v].r, m + 1, r, ql, qr);
}
};
===== Aniket =====
// Poly Prefix/Suffix hash
struct PolyHash {
    // Using an unsigned long long hash with
    base.
    static const unsigned long long BASE =
131ULL; // A common base value

```

```

    vector<unsigned long long> prefix,
suffix, power;
    int n;
    PolyHash(const string &s) {
        n = s.size();
        prefix.resize(n + 1, 0);
        suffix.resize(n + 2, 0);
        power.resize(n + 1, 0);
        power[0] = 1;
        for (int i = 0; i < n; i++) {
            power[i + 1] = power[i] * BASE;
        }
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] * BASE
+ (unsigned long long)(s[i]);
        }
        for (int i = n - 1; i ≥ 0; i--) {
            suffix[i + 1] = suffix[i + 2] *
BASE + (unsigned long long)(s[i]);
        }
    }
    unsigned long long getFwdHash(int L, int
R) {

```

```

        return prefix[R + 1] - prefix[L] *
power[R - L + 1];
    }
    unsigned long long getRevHash(int L, int
R) {
        return suffix[L + 1] - suffix[R + 2]
* power[R - L + 1];
    }
};
// Lazy - SegTree
class LazySegmentTree {
private:
    int n;
    vector<long long> tree, lazy;
    void build(vector<int> &arr, int node,
int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
        } else {
            int mid = (start + end) / 2;
            build(arr, 2 * node + 1, start,
mid);
            build(arr, 2 * node + 2, mid + 1,
end);

```

```

        tree[node] = tree[2 * node + 1] +
tree[2 * node + 2];
    }
}
void push(int node, int start, int end) {
    if (lazy[node] != 0) {
        tree[node] += (end - start + 1) *
lazy[node]; // Apply lazy
        if (start != end) {
            lazy[2 * node + 1] +=
lazy[node];
            lazy[2 * node + 2] +=
lazy[node];
        }
        lazy[node] = 0;
    }
}
void updateRange(int l, int r, int val,
int node, int start, int end) {
    push(node, start, end);
    if (r < start || l > end)
        return;
    if (l ≤ start && end ≤ r) {
        lazy[node] += val;

```

```

        push(node, start, end);
        return;
    }
    int mid = (start + end) / 2;
    updateRange(l, r, val, 2 * node + 1,
start, mid);
    updateRange(l, r, val, 2 * node + 2,
mid + 1, end);
    tree[node] = tree[2 * node + 1] +
tree[2 * node + 2];
}
long long queryRange(int l, int r, int
node, int start, int end) {
    push(node, start, end);
    if (r < start || l > end)
        return 0;
    if (l ≤ start && end ≤ r)
        return tree[node];
    int mid = (start + end) / 2;
    long long left = queryRange(l, r, 2 *
node + 1, start, mid);
    long long right = queryRange(l, r, 2
* node + 2, mid + 1, end);
    return left + right;
}

```

```

    }
public:
    LazySegmentTree(vector<int> &arr) {
        n = arr.size();
        tree.resize(4 * n);
        lazy.resize(4 * n);
        build(arr, 0, 0, n - 1);
    }
    void update(int l, int r, int val) {
updateRange(l, r, val, 0, 0, n - 1); }
    long long query(int l, int r) { return
queryRange(l, r, 0, 0, n - 1); }
};
// DSU
class DSU {
    vector<int> parent;

public:
    DSU(int n) {
        parent.resize(n + 1);
        iota(parent.begin(), parent.end(),
0);
    }
    int findparent(int n) {

```

```

        if (n == parent[n])
            return n;
        return parent[n] =
findparent(parent[n]);
    }
    void union(int u, int v) {
parent[findparent(u)] = findparent(v); }
};
// Sparse Table
class SparseTable {
private:
    vector<vector<pair<int, int>>> st;
    vector<int> log2_table;
    int n;
    pair<int, int> merge(pair<int, int> p1,
pair<int, int> p2) {
        if (p1.first > p2.first)
            return p1;
        if (p1.first < p2.first)
            return p2;
        return {p1.first, min(p1.second,
p2.second)};
    }

```

```

public:
    SparseTable(vector<pair<int, int>> &arr)
    {
        n = arr.size();
        int max_log = log2(n) + 1;
        st.assign(n, vector<pair<int,
int>>(max_log));
        log2_table.resize(n + 1);
        log2_table[1] = 0;
        for (int i = 2; i ≤ n; i++)
            log2_table[i] = log2_table[i / 2]
+ 1;
        for (int i = 0; i < n; i++)
            st[i][0] = arr[i];
        for (int j = 1; (1 << j) ≤ n; j++) {
            for (int i = 0; i + (1 << j) ≤
n; i++) {
                st[i][j] = merge(st[i][j -
1], st[i + (1 << (j - 1))][j - 1]);
            }
        }
        pair<int, int> query(int L, int R) {
            int length = R - L + 1;

```

```

        int k = log2_table[length];
        return merge(st[L][k], st[R - (1 <<
k) + 1][k]);
    }
};

vector<int> fact(N, 1);
int binpow(int a, int b, int m) {
    a %= m;
    int res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}

int modmuli(int a, int b, int p) {
    long long res = 0;
    a %= p;
    while (b) {
        if (b & 1)
            res = (res + a) % p;
        a = (2 * a) % p;
    }
}

```

```

        b >>= 1;
    }
    return res;
}

int gcdExtended(int a, int b, int *x, int *y)
{
    if (a == 0) {
        *x = 0, *y = 1;
        return b;
    }
    int x1, y1;
    int gcd = gcdExtended(b % a, a, &x1,
&y1);
    *x = y1 - (b / a) * x1;
    *y = x1;
    return gcd;
}

int modInverse(int b, int m) {
    int x, y;
    int giga = gcdExtended(b, m, &x, &y);
    if (giga != 1)
        return -1;
    return (x % m + m) % m;
}

```

```

void factorial(int p) {
    for (int i = 2; i < N; i++) {
        fact[i] = modmuli(fact[i - 1], i, p);
    }
}

int nCr(int n, int r, int p) {
    if (r > n)
        return 0;
    return modmuli(fact[n],
modInverse(modmuli(fact[n - r], fact[r], p),
p), p);
}

int modDivide(int a, int b, int m) {
    a = a % m;
    int inv = modInverse(b, m);
    if (inv == -1)
        return 0;
    else
        return (inv * a) % m;
}

vector<int> sieve() {
    vector<int> prime(N + 1, 1);
    for (int p = 2; p * p ≤ N; p++) {

```

```

        if (prime[p] == true) {
            for (int i = p * p; i ≤ N; i +=
p)
                prime[i] = false;
        }
    }
    vector<int> ans;
    for (int i = 2; i ≤ N; i++) {
        if (prime[i]) {
            ans.push_back(i);
        }
    }
    return ans;
}

// Disjoint Set Union
class DSU {
public:
    int vtx;
    vector<int> prnt, rank, size;
    DSU(int n) {
        vtx = n;
        make(n);
    }
    void make(int n) {

```

```

    prnt.resize(n);
    rank.resize(n, 0);
    size.resize(n, 1);
    for (int i = 0; i < n; i++) {
        prnt[i] = i;
    }
}

int find(int u) {
    if (u == prnt[u]) {
        return u;
    }
    return prnt[u] = find(prnt[u]);
}

void unionRank(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
        if (rank[u] > rank[v]) {
            swap(u, v);
        }
        prnt[u] = v;
        if (rank[u] == rank[v]) {
            rank[v]++;
        }
    }
}

```

```

    }
}

void unionSize(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
        if (size[u] > size[v]) {
            swap(u, v);
        }
        prnt[u] = v;
        size[v] += size[u];
    }
}

};

vector<int> spf;
void sieve() {
    spf.resize(N + 1, 1);
    spf[0] = 1;
    for (int i = 2; i ≤ N; i++) {
        if (spf[i] == 1) {
            for (int j = i; j ≤ N; j += i) {
                if (spf[j] == 1) {
                    spf[j] = i;
                }
            }
        }
    }
}

```

```
        }
    }
}
map<int, int> getFactorization(int x) {
    map<int, int> ret;
    if (x ≤ 0)
        return ret;
    while (x ≠ 1) {
        ret[spf[x]]++;
        x = x / spf[x];
    }
    return ret;
}
```